Teoría del Error, serie de Taylor y Raíces de Funciones

1 Introducción

En este laboratorio se desarrollará una aplicación con interfaz gráfica (GUI) que permita estudiar el comportamiento numérico de distintos métodos para la resolución de ecuaciones no lineales, así como el análisis de error y aproximaciones mediante Series de Taylor.

Como primera etapa, cada equipo de estudiantes (máximo tres) deberá construir en MATLAB un conjunto de funciones reutilizables para cada método expuesto en clase. La intención es separar con claridad:

- La formulación matemática del método.
- La implementación computacional.
- El análisis de resultados (convergencia, divergencia y sensibilidad).

El objetivo no es únicamente "obtener una raíz", sino analizar el comportamiento frente a distintos escenarios: convergencia, divergencia, sensibilidad a la semilla inicial (condiciones iniciales) y efecto de la multiplicidad algebraica.

2 Objetivos

1. Aplicar correctamente las definiciones de error absoluto, relativo y porcentual.
2. Diseñar funciones en MATLAB con entradas y salidas claramente definidas para cada procedimiento numérico.
3. Implementar la aproximación de una función mediante Serie de Taylor y analizar el error de truncamiento.
4. Implementar métodos cerrados y abiertos para el cálculo de raíces.
5. Analizar el comportamiento del método de Punto Fijo (casos de convergencia y divergencia).
6. Estudiar el efecto de raíces con multiplicidad algebraica mayor que uno.
7. Comparar convergencia entre Newton clásico y Newton modificado.

3 Recordatorio Conceptual

Conceptos Clave

Errores. Sea x_{real} el valor verdadero y x_{aprox} una aproximación.

- Error absoluto: $E_T = |x_{\text{real}} - x_{\text{aprox}}|$.
- Error relativo: $\epsilon_T = \dfrac{|x_{\text{real}} - x_{\text{aprox}}|}{|x_{\text{real}}|}$ (si $x_{\text{real}} \neq 0$).
- Error porcentual: $\epsilon_p = 100\,\epsilon_r$ (en %).

Serie de Taylor. Aproximación alrededor de a hasta orden n:

$$f_n(x) = \sum_{k=0}^{n} \frac{f^{(k)}(a)}{k!}(x-a)^k.$$

Métodos cerrados. Requieren un intervalo $[a, b]$ con cambio de signo, es decir, $f(a)f(b) < 0$.
Métodos abiertos. Requieren una o dos aproximaciones iniciales.
Punto fijo. Transformar $f(x) = 0$ en $x = g(x)$ y verificar localmente $|g'(x)| < 1$ para convergencia.
Raíz múltiple.
Newton modificado (multiplicidad conocida).

$$x_{k+1} = x_k - m\,\frac{f(x_k)}{f'(x_k)}.$$

y el método para raices multiples de Ralston-Rabinowitz que se optiene al sustituir $f(x)$ por $u(x) = f(x)/f'(x)$ en el método de Newton clásico.

4 Primera Parte: Construcción de Funciones en MATLAB

Antes de desarrollar la GUI, el laboratorio inicia con la implementación modular de funciones para cada método visto en clase. Cada algoritmo debe quedar encapsulado en un archivo independiente de MATLAB, con formulación clara de entradas, salidas y criterios de parada.

Requisito común para todas las funciones de raíces

Cada método debe retornar como mínimo:

- La aproximación final `raiz`.
- El número de iteraciones `iter`.
- Una `tabla` (historial iterativo).
- Un indicador `converge` (1 si converge, 0 si no).

Criterios de parada obligatorios (use ambos):

- Por cantidad de iteraciones $i \leq$ `maxIter`.
- Por tolerancia de error: $\epsilon_x <$ `tol`.

1. Funciones para análisis de error

- Implementar funciones para error absoluto, error relativo y error porcentual.
- Entradas: valor real y valor aproximado.
- Si el valor real es cero, justificar el tratamiento del error relativo para evitar división por cero.

2. Función para Serie de Taylor

- Implementar una función que reciba: función, punto de expansión a, punto de evaluación x, y orden n.
- Retornar: polinomio truncado, aproximación numérica y (si aplica) error.

3. Funciones para métodos de raíces

Implementar una función por cada método visto en clase:

- Bisección
- Regla falsa
- Punto fijo
- Newton-Raphson
- Secante
- Newton modificado (raíces múltiples)

4. Claridad sobre las formulaciones

Para evitar implementaciones ambiguas, en el informe y en el código debe quedar clara la firma de cada método:

- **Bisección y Regla Falsa:** `metodo(f,a,b,tol,maxIter)`.
- **Punto Fijo:** `puntoFijo(g,x0,tol,maxIter)` (dejar explícito $x = g(x)$).
- **Newton-Raphson:** `newton(f,df,x0,tol,maxIter)`.
- **Secante:** `secante(f,x0,x1,tol,maxIter)`.
- **Newton modificado:** `newtonMultiple(f,df,x0,m,tol,maxIter)`.

5 Planteamiento del Problema: Planificación y uso de métodos numéricos en un contexto de Ingeniería Civil

En ingeniería civil es común enfrentarse a modelos donde una magnitud física (deflexión, pendiente, momento flector, esfuerzo, energía potencial, etc.) se describe mediante una función continua $y(x)$ en un dominio espacial $x \in [a, b]$. A partir de dicha función se plantean condiciones de diseño y verificación que, en muchos casos, se reducen a resolver ecuaciones del tipo:

$$f(x) = 0, \qquad f'(x) = 0, \qquad f''(x) = 0,$$

donde x representa una variable de interés del problema (posición a lo largo de un elemento estructural, parámetro geométrico, valor de una carga equivalente, etc.).

Contexto sugerido (validable con un docente)

Considere un elemento estructural idealizado (por ejemplo, una viga, una losa en una dirección, un elemento de fundación simplificado o un modelo de deformación) en el que se defina una función $F(x)$ asociada a un criterio de diseño. Dependiendo del enfoque del curso que se esté articulando con el docente, $F(x)$ puede representar:

- una condición de **equilibrio** (resultante nula),
- una condición de **servicio** (deflexión objetivo, desplazamiento máximo permitido),
- una condición de **óptimo estructural** (máximo/mínimo de una respuesta),
- o un cambio de **concavidad** (punto de inflexión asociado a transición de comportamiento).

La formulación exacta de $F(x)$ debe ser consultada y validada con un docente del programa (por ejemplo, un docente de Estructuras, Resistencia de Materiales, Estática, Dinámica o Física), con el fin de asegurar que el modelo represente un fenómeno plausible y que los supuestos sean coherentes.

Interpretación física de las ecuaciones

Una vez definida la función de interés $F(x)$, se realizarán tres análisis numéricos, cada uno con un significado físico distinto:

1. **Caso A: Condición de cruce / equilibrio**
   Resolver $F(x) = 0$ en un intervalo $[a, b]$.
   - **Interpretación típica:** posición donde una magnitud se anula (por ejemplo, desplazamiento cero, fuerza neta cero, momento igual a cero, cumplimiento exacto de una condición de diseño).
   - **Condición suficiente para métodos cerrados (Bisección/Regla Falsa):** F continua en $[a, b]$ y $F(a)\,F(b) < 0$.
   - **Condiciones para métodos abiertos (Newton/Secante/Punto fijo):** existencia de semilla(s) inicial(es) en una vecindad de la raíz y derivadas no degeneradas (por ejemplo, $F'(x) \neq 0$ cerca de la raíz para Newton).

2. **Caso B: Condición de extremo (máximo o mínimo)**
   Resolver $F'(x) = 0$ en un intervalo $[a, b]$.

   - **Interpretación típica:** localización de un máximo/mínimo de la respuesta (por ejemplo, deflexión máxima, esfuerzo máximo, momento máximo).
   - **Condición necesaria:** $F'(x^\star) = 0$ (con $x^\star$ punto candidato).
   - **Condición suficiente para clasificar el extremo:** si $F''(x^\star) > 0$ hay mínimo local, si $F''(x^\star) < 0$ hay máximo local.
   - **Condición suficiente para métodos cerrados aplicados a $G(x) = F'(x)$:** F' continua en $[a, b]$ y $F'(a)\,F'(b) < 0$.

3. **Caso C: Condición de inflexión / cambio de concavidad**
   Resolver $F''(x) = 0$ en un intervalo $[a, b]$.

   - **Interpretación típica:** punto donde cambia la concavidad de la respuesta; en algunos modelos está asociado a transición de régimen o redistribución del comportamiento.
   - **Condición necesaria:** $F''(x^\dagger) = 0$.
   - **Condición suficiente de inflexión:** F'' cambia de signo alrededor de $x^\dagger$ (por ejemplo, existe $\varepsilon > 0$ tal que $F''(x^\dagger - \varepsilon)\,F''(x^\dagger + \varepsilon) < 0$).
   - **Condición suficiente para métodos cerrados aplicados a $H(x) = F''(x)$:** F'' continua en $[a, b]$ y $F''(a)\,F''(b) < 0$.

Planificación numérica (qué método usar y por qué)

Para cada caso, el estudiante debe:

1. **Definir el dominio** $[a, b]$ de acuerdo con el contexto físico (por ejemplo, longitud de una viga, rango admisible del parámetro, límites geométricos). Explicar qué representa a, qué representa b y por qué ese intervalo tiene sentido en ingeniería.

2. **Verificar condiciones previas** antes de aplicar el método:

   - continuidad de la función objetivo (para métodos cerrados),
   - cambio de signo en el intervalo (si se usa método cerrado),
   - derivada no nula y semilla razonable (si se usa Newton),
   - denominadores no degenerados (si se usa secante),
   - condición local de convergencia en Punto Fijo (si se propone $x = g(x)$).

3. **Seleccionar el método numérico** (cerrado u abierto) y justificar la elección en función de:

   - garantías de convergencia,
   - costo computacional,
   - sensibilidad a condiciones iniciales,
   - robustez frente a raíces múltiples.

4. **Definir tolerancia y máximo de iteraciones** en términos del problema (unidades y precisión requerida).

Modelo de referencia para el laboratorio

Como caso guía (base para comparar métodos), se propone la función:

$$f(x) = x(L^3 - 2Lx^2 + x^3), \qquad L = 6,$$

y se solicita analizar en el intervalo $[0, L]$ los tres casos:

$$f(x) = 0, \qquad f'(x) = 0, \qquad f''(x) = 0.$$

Nota de análisis: En $[0, L]$ la función presenta raíces con distinta multiplicidad. Identifique cuáles son raíces simples y cuáles son múltiples, y discuta cómo afecta esto la convergencia, especialmente en métodos abiertos.

Además, se analizará la función:

$$h(x) = (x - 2)^2(x + 1),$$

para estudiar de forma explícita el comportamiento numérico ante una raíz doble y comparar Newton clásico vs. Newton modificado.

Consulta docente (requisito sugerido)

Se recomienda (o se deja como requisito, según criterio del docente) una breve consulta con un profesor del área (Estructuras/Física/Resistencia de Materiales) para validar:

- el significado físico de $F(x)$,
- el rango $[a, b]$ y parámetros involucrados,
- la interpretación de resolver $F(x) = 0$, $F'(x) = 0$ y $F''(x) = 0$ en el contexto propuesto.

6 Requerimientos de la Aplicación

La aplicación debe estar organizada en pestañas:

Etapa previa obligatoria

- Construcción y validación de las funciones en MATLAB antes de la interfaz.
- Prueba individual de cada función con ejemplos conocidos.
- Verificación de que las formulaciones implementadas coincidan con las expresiones teóricas.

P1 – Errores y Taylor

- Uso de funciones de error y Serie de Taylor.
- Entrada: función, punto a, orden n y punto de evaluación.
- Comparación entre valor real y aproximación.

P2 – Método Gráfico

- Visualización de $f(x)$, $f'(x)$ y $f''(x)$.
- Selección interactiva de intervalo.

P3 – Métodos Cerrados

- Integración de Bisección y Regla Falsa.
- Tabla de iteraciones y criterio de parada.
- Validación: exigir $f(a)f(b) < 0$.

P4 – Métodos Abiertos

- Integración de Newton-Raphson, Secante, Punto Fijo y Newton modificado.
- Para Newton: validar derivada no nula (o muy pequeña).
- Para Secante: validar denominador no nulo.

En Newton modificado, permitir ingresar la multiplicidad m.

7 Análisis Requerido

El informe debe incluir:

1. Comparación de número de iteraciones entre métodos cerrados y abiertos.
2. Un caso donde Punto Fijo converge y otro donde diverge (justificado).
3. Comparación Newton clásico vs Newton modificado usando $h(x)$.
4. Discusión del efecto de la multiplicidad en la velocidad de convergencia.
5. Discusión de sensibilidad a la semilla inicial (al menos dos semillas por método abierto).

8 Entregables

- Informe en PDF con análisis matemático y numérico.
- Carpeta con las funciones en MATLAB (una por método), nombradas y comentadas.
- Código fuente de la GUI, comentado y organizado.
- Video demostrativo (2–3 minutos).

9 Criterios de Evaluación

La evaluación del laboratorio no se centrará únicamente en la obtención de resultados numéricos, sino en el dominio conceptual, la implementación técnica y la capacidad de análisis y argumentación del estudiante.

1. Dominio conceptual (25 %)

Se evaluará:

- Comprensión de las ecuaciones $f(x) = 0$, $f'(x) = 0$ y $f''(x) = 0$ en el contexto del problema.
- Identificación correcta de raíces simples y múltiples.
- Justificación adecuada del método numérico seleccionado.
- Comprensión de las condiciones necesarias y suficientes para aplicar cada método.

2. Implementación y estructura del programa (25 %)

Se evaluará:

- Modularidad del código (funciones independientes).
- Correcta implementación de criterios de parada.
- Manejo de casos especiales (derivada cercana a cero, división por cero, falta de cambio de signo).
- Claridad, organización y comentarios en el código.

3. Análisis del error y comportamiento numérico (25 %)

Se evaluará:

- Uso adecuado de error absoluto, relativo y porcentual.
- Comparación entre métodos cerrados y abiertos.
- Discusión del efecto de la multiplicidad en la convergencia.
- Análisis de sensibilidad respecto a la semilla inicial.

4. Defensa y argumentación (15 %)

Durante la sustentación (individual o grupal) se evaluará:

- Capacidad de explicar el comportamiento del método.
- Interpretación física del resultado.
- Respuesta a preguntas técnicas del docente.

5. Coherencia integral del trabajo (10 %)

Se evaluará:

- Consistencia entre teoría, código y análisis.
- Presentación clara del informe.
- Relación explícita con el contexto de ingeniería civil.

Resumen porcentual

Dominio conceptual	25 %
Implementación del programa	25 %
Análisis del error	25 %
Defensa oral	15 %
Coherencia integral	10 %

10 Ejemplo de Estructura de Código

El siguiente script hace el llamado de la función para Newton modificado:

```
% Definición de la función y su derivada
f  = @(x) (x - 10) * (x - pi)^2;
df = @(x) (x - pi)^2 + 2 * (x - 10) * (x - pi);

% Configuración de parámetros para buscar la raíz x = pi
x0        = 3.5;     % Punto inicial cercano a pi (3.1415...)
m         = 2;       % Multiplicidad de la raíz (x - pi)^2
tol       = 1e-10;
maxIter   = 100;

% Ejecución
[raiz, tabla, iter, converge] = newtonMultiple(f, df, x0, m, tol,
    maxIter);

% Resultados
fprintf('Raíz hallada: %.10f (Valor real: %.10f)\n', raiz, pi);
disp('Tabla de iteraciones:');
disp(tabla);
```

Listing 1: Ejemplo de llamada a la función de Newton modificado

Función de Newton modificado para raíces múltiples conocida la multiplicidad m:

```
function [raiz, tabla, iter, converge] = newtonMultiple(f, df, x0, m,
    tol, maxIter)
% newtonMultiple: Modified Newton for known multiplicity roots
% tabla: [k, x, f(x), errX, errF]

x = x0;
tabla = zeros(maxIter, 5);
converge = 0;

for k = 1:maxIter
    fx  = f(x);
    dfx = df(x);

    if abs(dfx) < 1e-14
        iter = k;
        raiz = x;
        tabla = tabla(1:k-1,:);
        return
    end

    xNuevo = x - m * (fx / dfx);
    errX = abs(xNuevo - x);
    errF = abs(f(xNuevo));

    tabla(k,:) = [k, xNuevo, f(xNuevo), errX, errF];

```

```
    if (errX < tol) || (errF < tol)
        raiz = xNuevo;
        iter = k;
        converge = 1;
        tabla = tabla(1:k,:);
        return
    end

    x = xNuevo;
end

raiz = x;
iter = maxIter;
tabla = tabla(1:maxIter,:);
end
```

Listing 2: Estructura sugerida para Newton modificado en MATLAB

11 Bibliografía

Referencias

[1] R. L. Burden, J. D. Faires, and A. M. Burden, *Numerical Analysis*, 10th ed. Cengage Learning, 2015.

[2] D. R. Kincaid and E. W. Cheney, *Numerical Analysis: Mathematics of Scientific Computing*. Brooks/Cole, 1991.

[3] F. McKenna, G. L. Fenves, and M. H. Scott, "Opensees: a framework for earthquake engineering simulation," *Journal of Computing in Civil Engineering (ASCE)*, 2010.

[4] S. Mazzoni, F. McKenna, M. H. Scott, and G. L. Fenves, *OpenSees Command Language Manual*, OpenSees Project, 2006.

[5] MathWorks, "fzero — root of nonlinear function (matlab documentation)," Online documentation.

[6] ——, "fsolve — solve system of nonlinear equations (matlab documentation)," Online documentation.

[7] V. V. Tuhaise *et al.*, "Technologies for digital twin applications in construction," *Automation in Construction*, 2023.

[8] M. Deng, C. C. Menassa, and V. R. Kamat, "From bim to digital twins: a systematic review of the built environment," *ITcon*, 2021.